

Indexing in DBMS

1. Overview

Indexing is a technique used to improve database query performance by reducing the number of disk reads needed to find data.

Instead of scanning the full table, the database uses an **index data structure** to quickly locate rows.

None

Without Index:

Full table scan

With Index:

Fast lookup path

2. Simple Analogy

Like a book index:

None

Word → Page Number

Database index:

None

Value → Row location

Instead of reading the whole book/table.

3. Why Indexing Is Needed

Without index:

SQL

```
SELECT * FROM users WHERE email='a@x.com';
```

Database may scan millions of rows.

With index:

None

Go directly to matching row

Much faster.

4. Core Components of Index

A. Search Key

Field used for searching.

Can be:

- Primary key
- Candidate key
- Foreign key
- Frequently queried column

Examples:

None

`user_id`

`email`

`phone`

`created_at`

B. Data Reference

Pointer/reference to actual row location.

None

`key → disk block / page / row id`

5. Common Example

Users table:

id	name	email
1	Sam	a@mail.com
2	Ansh	b@mail.com

Index on email:

None

a@mail.com → row 1

b@mail.com → row 2

6. Benefits of Indexing

Faster SELECT queries

Faster WHERE filters

Faster JOINS

Faster ORDER BY (sometimes)

Faster GROUP BY (sometimes)

Faster range search

7. Typical Queries Improved

SQL

```
SELECT * FROM users WHERE id=5;
```

```
SELECT * FROM orders
```

```
WHERE user_id=101;
```

```
SELECT * FROM logs
```

```
WHERE created_at > '2026-01-01';
```

8. Index Is Optional

Tables work without indexes.

But performance can degrade badly for large data.

None

Small table → maybe okay

Large table → index important

9. Primary vs Secondary Access

- **Primary access** = actual table data
- **Secondary access** = index path to reach data

Index does not replace table data.

It helps find rows faster.

10. Is Index Always Sorted?

Many classical indexes are sorted (especially B-Tree based).

Examples:

- B-Tree index → sorted
- Clustered index → sorted storage order

But not all indexes are sorted:

- Hash indexes are not sorted.

So better statement:

None

Many indexes are ordered, but not all.

11. Common Types of Indexes

A. B-Tree Index (Most Common)

Used in:

- PostgreSQL
- MySQL

Good for:

- Equality search
- Range search
- Sorting

B. Hash Index

Good for exact lookup:

None

```
WHERE id = 5
```

Not good for ranges.

C. Composite Index

Multiple columns:

SQL

```
INDEX(user_id, created_at)
```


D. Full Text Index

For search engines / text matching.

12. Trade-offs of Indexing

Advantages

Faster reads

Disadvantages

Extra storage

Slower INSERT/UPDATE/DELETE

More maintenance

Too many indexes hurt writes

13. Example Trade-off

Table receives 10 million writes/day.

Too many indexes means each insert updates many index structures.

Write latency increases.

14. Interview Design Tip

Index columns used in:

- WHERE
- JOIN

- ORDER BY
- GROUP BY
- High-frequency lookups

15. Bad Indexing Practices

Index every column

Index low-cardinality columns blindly

Ignore write-heavy workload

Duplicate indexes

No monitoring/query plans

16. Clustered vs Non-Clustered

Type	Meaning
Clustered	Table rows stored in index order
Non-clustered	Separate structure with pointers

17. Real Example

SQL

```
CREATE INDEX idx_users_email  
  
ON users(email);
```

Now:

```
SQL
SELECT * FROM users
WHERE email='a@mail.com';
```

Becomes much faster.

18. HLD Perspective

For large systems:

```
None
Users DB
Orders DB
Search DB
Analytics DB
```

Proper indexing is critical for latency SLA.

19. Short Revision Notes

None

Indexing =

Extra data structure for fast lookup

key → row location

Benefits:

fast SELECT / WHERE / JOIN

Costs:

extra storage

slower writes

20. One-Line Summary

Indexing improves database read performance by maintaining a searchable structure that maps key values to row locations, reducing expensive full-table scans.